



# UNIVERSITÀ DEGLI STUDI DI MESSINA

DIPARTIMENTO DI SCIENZE MATEMATICHE E INFORMATICHE,  
SCIENZE FISICHE E SCIENZE DELLA TERRA

Master's Degree in Data Science

---

## SmartFood - Blockchain-Based Supply Chain Validation

Domenico  
Villari  
Student ID 574194

---

ACADEMIC YEAR 2024/2025

# 1 Introduction

This project presents an innovative blockchain-based solution for agri-food supply chain management. By leveraging the Circular Protocol blockchain as a public ledger, the system enables transparent and immutable tracking of products from origin to consumer.

The solution addresses critical challenges in food traceability, including data fragmentation, lack of transparency, and verification difficulties. The implementation features a JavaScript frontend and FastAPI backend, providing stakeholders with an intuitive interface to record, verify, and access supply chain data.

This approach demonstrates how blockchain technology can enhance trust, ensure data integrity, and facilitate comprehensive traceability in agri-food supply chains.

# 2 Circular Blockchain

Circular is a Layer-1 decentralized operating system for regulated domains (healthcare, pharmaceuticals, scientific research, AI) that provides an immutable, auditable data layer designed for cross-organizational collaboration without compromising compliance or privacy. Its architecture supports verifiable records at scale and offers SDKs/APIs for rapid integration with existing systems. The platform emphasizes compliance readiness (e.g., HIPAA, GDPR, ICH GCP) and supports deployments with private chains and public validation to deliver continuous evidence for regulators.

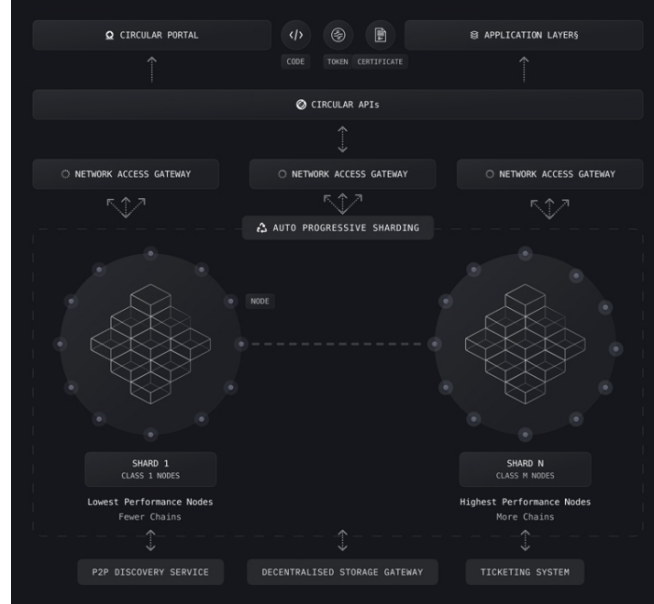


Figura 1: Circular Architecture [1]

## 2.1 Interoperability and co-chains

Circular natively supports multiple co-chains on the same node and cross-chain references, allowing sensitive operations to remain private while anchoring public proofs for auditability and provenance across jurisdictions. Nodes can mint blocks on several chains in parallel, enabling selective participation and visibility per chain while preserving verifiable cross-chain integrity for end-to-end supply-chain traceability.

## 2.2 Node and storage model

Each node integrates P2P messaging, transaction routing, ticketed block assembly, and a consensus engine that compiles signed peer reports; approved blocks are persisted by a Disk Manager. Storage is pluggable—append-only files for low overhead or a local SQL database for indexed queries (e.g., by address and block range), supporting timelines and audits in application backends.

## 2.3 Proof of Reputation (PoR)

**Proof of Reputation (PoR)** is a consensus mechanism that complements conventional models such as Proof of Work (PoW) and Proof of Stake (PoS). Unlike these approaches, which rely primarily on computational power or financial stake, PoR evaluates the reputation of participants **based on their historical actions**, contributions, and interactions.

In the Circular Blockchain, **node rewards** are calculated as the **product of transaction fees and reputation**. Reputation increases when nodes perform reliably and align with the majority consensus, and decreases when they deviate or behave unreliably.

During consensus, the initiating node aggregates peer responses to finalize outcomes. Nodes that consistently demonstrate reliability and truthfulness strengthen their credibility, while those exhibiting suspicious or anomalous behavior gradually lose reputation and may ultimately be excluded from the network.

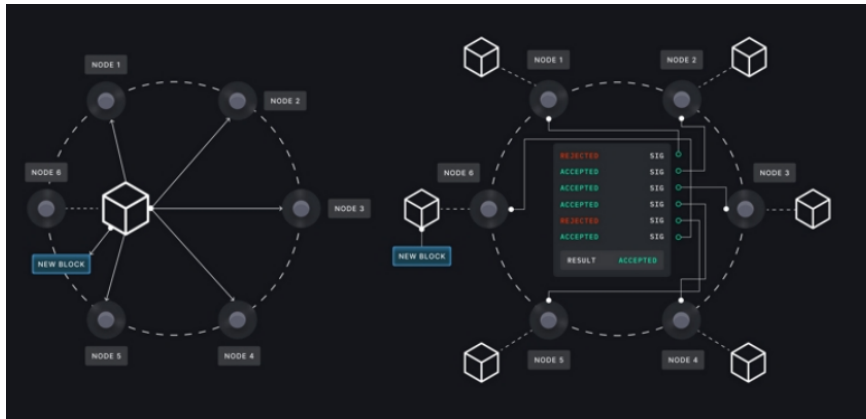


Figura 2: a)Block broadcasting - b)Consensus Report Filing [1]

### 3 System Architecture

The system consists of a **FastAPI backend**, a **JavaScript** single-page frontend, integration with the **Circular SDK** to manage the transaction lifecycle, and a **MySQL database** used for auxiliary state, QR-code storage, and certificate export metadata.

Security is enforced through **JSON Web Tokens (JWTs)** issued as **bearer tokens**. In this model, authenticated clients receive a signed JWT containing claims such as subject, roles, and expiration time. The token must be presented on protected routes, and the backend validates its integrity, authenticity, and scope before granting access. Since bearer tokens grant access to whoever possesses them, they are configured with short expiration times (30 minutes) and renewal mechanisms to reduce the risk of misuse.

On the client side, private keys are encrypted and stored locally using **IndexedDB**. Key material is derived using **PBKDF2 (Password-Based Key Derivation Function 2)** with a passkey, and decryption for transaction signing is performed only on demand. The actual encryption scheme employs **AES-GCM (Advanced Encryption Standard in Galois/Counter Mode)**, a widely adopted **symmetric cipher** that ensures both confidentiality (preventing unauthorized disclosure) and integrity (detecting tampering). This design enables secure, client-side key management while maintaining efficient, standards-based authentication and authorization on the backend.

### 4 Authentication: Login, Register, and Google Sign-in

Access begins on the Login page, where user credentials are securely submitted to a protected backend endpoint. Upon successful verification, performed by comparing the provided password against the stored hash and matching the registered email in the MySQL database, a JWT is issued. This token is then stored in `localStorage`, and the browser is redirected to the dashboard to establish the authenticated session state.

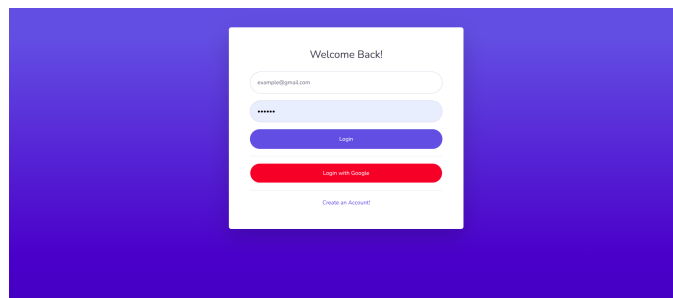


Figura 3: Login Page

New users are onboarded via the Register page, which performs client-side validation (email format and password confirmation), submits identity, role, and wallet address to the backend, and, if successful, mirrors the login flow by issuing a token and redirecting to the dashboard.

Figura 4: Register Page

Beyond password-based flows, the system supports federated authentication via **Google using OpenID Connect**; the application registers a Google client with scope `openid email profile`, exposes a redirect endpoint to Google, and handles the callback to obtain the user's OIDC claims, after which it creates or updates the local profile and issues a first-party JWT before redirecting to the dashboard.

Specifically, the backend defines `/auth/google/login` to initiate the external authorization request and `/auth/google/callback` to process the response, extract `userinfo` (email, name, picture, subject), upsert the user with `google_id` and `profile_picture`, and generate a session token for subsequent requests.

```

1 # Callback Google
2 @app.get("/auth/google/callback")
3 async def google_callback(request: Request, response_class=Token):
4     '''Gestisce il callback da Google e crea/autentica l'utente.
5     -Args:
6         request (Request): La richiesta HTTP corrente contenente il
7         token di accesso.
8     -Returns:
9         RedirectResponse: Reindirizza alla dashboard con il token JWT
10        impostato come cookie.
11    -Raises:
12        HTTPException: Se non riesce a ottenere le informazioni utente
13        da Google o se si verifica un errore durante l'autenticazione.'''
14
15    try:
16        token = await oauth.google.authorize_access_token(request)
17        user_info = token.get('userinfo')
18
19        if not user_info:
20            raise HTTPException(status_code=400, detail="Impossibile
21            ottenere informazioni utente da Google")

```

```

22     # Cerca o crea utente nel database
23     google_user = GoogleUser(
24         email=user_info.get('email'),
25         name=user_info.get('name'),
26         picture=user_info.get('picture'),
27         google_id=user_info.get('sub')
28     )
29
30     # Controlla se l'utente esiste gia' nel database
31     conn = get_db_connection()
32     cursor = conn.cursor(dictionary=True)
33
34     cursor.execute("SELECT * FROM user WHERE email=%s", (
35         google_user.email,))
36     existing_user = cursor.fetchone()
37
38     if not existing_user:
39         # Crea nuovo utente
40         cursor.execute("""
41             INSERT INTO user (nome, email, ruolo, wallet_addr,
42             google_id, profile_picture)
43             VALUES (%s, %s, %s, %s, %s, %s)
44             """, (
45                 google_user.name,
46                 google_user.email,
47                 "produttore", # ruolo default
48                 "", # wallet_addr vuoto per ora
49                 google_user.google_id,
50                 google_user.picture
51             ))
52         conn.commit()
53         user_id = cursor.lastrowid
54     else:
55         user_id = existing_user['id']
56         # Aggiorna informazioni Google se necessario
57         cursor.execute("""
58             UPDATE user SET google_id=%s, profile_picture=%s WHERE
59             id=%s
60             """, (google_user.google_id, google_user.picture, user_id)
61         )
62         conn.commit()
63
64     cursor.close()
65     conn.close()
66
67     # Crea JWT token per la sessione
68     token_data = {"sub": google_user.email}
69     token = create_access_token(token_data)
70
71     html_content = f"""
72     <html>
73     <head>
74     <script>
75         localStorage.setItem("token", "{token}");
76         window.location.href = "/dashboard";
77     </script>
78     </head>
79     <body>
80         Accesso eseguito, reindirizzamento...
81     </body>

```

```

78         </html>
79         """
80         return HTMLResponse(content=html_content)
81
82     except Exception as e:
83         raise HTTPException(status_code=400, detail=f"Errore durante l'
'autenticazione Google: {str(e)}")

```

Listing 1: API Endpoint for the login procedure with Google

## 5 Dashboard

The Dashboard serves as the primary consolidated entry point for authenticated users, providing an overview of profile information and navigation to core operational and verification functions. Acting as a centralized hub, it enables users to initiate new supply-chain steps, explore product lineage through the *Filiera* module, and review recent activity within the *Timeline* interface. Additionally, users may access wallet management and key provisioning pages to maintain secure cryptographic elements.

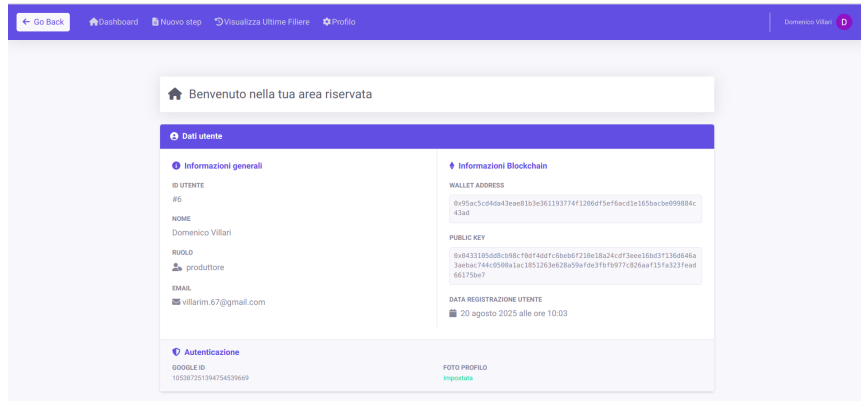


Figura 5: Dashboard Page showcasing profile overview and navigation options.

Consistent with the security model employed across the application, the Dashboard page incorporates frontend protection scripts that verify authentication state before granting content access. This ensures that only validly authenticated users—bearing unexpired JWT tokens—can interact with protected resources, reinforcing session integrity in the single-page application architecture.

Below is a representative excerpt from `protected.js`, illustrating the core functions that validate the bearer token with the backend and redirect unauthenticated users to the login page:

```

1 async function validateToken() {
2     const token = localStorage.getItem("token");
3     if (!token) return false;
4
5     try {
6         const response = await fetch(`${API_BASE}/auth/validate_token
`, {

```

```

7         method: "GET",
8         headers: {
9             "Authorization": `Bearer ${token}`
10        }
11    });
12
13    return response.ok; // true if 200, false if 401 or other
14    error
15  } catch (error) {
16    console.error("Token validation error:", error);
17    return false;
18  }
19
20  async function checkAuth() {
21    const isValid = await validateToken();
22    if (!isValid) {
23      // Remove invalid or missing token and redirect to login
24      localStorage.removeItem("token");
25      window.location.href = `${API_BASE}/login`;
26      return false;
27    }
28    return true;
29  }

```

Listing 2: Key authentication validation functions in `protected.js`

This pattern exemplifies a secure client-side guard mechanism, critical to preserving protected workflows and sensitive data visibility within the web application.

## 5.1 Secure Key Provisioning

Before users can initiate certified supply-chain transactions, they must provision signing capability locally to safeguard cryptographic keys and ensure secure signing operations. This critical step is performed through the *Insert Private Key* page, which enforces client-side key management and encryption protocols to avoid transmission of secret keys to the server or exposure during transit.

Figura 6: Insert Private Key page

Users input their raw private key (in hexadecimal or base64 form) and select a passphrase used to derive an **AES-GCM** encryption key via **PBKDF2** with a high iteration count (200,000), enhancing resistance against brute-force assaults.

The private key is then encrypted entirely within the browser using the Web Crypto API's AES-GCM implementation to guarantee confidentiality and integrity of the stored key material.

Storage occurs in **IndexedDB** under the **wallet-db** database and the **encrypted\_keys** object store keyed by the user's normalized wallet address. This structured key-value store enables efficient retrieval and enforces cryptographic separation of key material per wallet identity. Upon successful encryption and storage, the in-memory private key bytes are zeroed to limit residual exposure.

The asynchronous JavaScript logic managing this procedure is summarized as follows:

```

1  async function encryptPrivateKey(privateKeyUint8, passphrase) {
2    const salt = crypto.getRandomValues(new Uint8Array(16));
3    const iv = crypto.getRandomValues(new Uint8Array(12));
4    const aesKey = await deriveKey(passphrase, salt); // PBKDF2 key
      derivation with 200k iterations
5    const ciphertext = await crypto.subtle.encrypt(
6      { name: "AES-GCM", iv: iv },
7      aesKey,
8      privateKeyUint8
9    );
10   return {
11     ciphertext: arrayBufferToBase64(ciphertext),
12     salt: arrayBufferToBase64(salt.buffer),
13     iv: arrayBufferToBase64(iv.buffer),
14     version: "1.0",
15     created_at: new Date().toISOString()
16   };
17 }
18
19 document.getElementById("importKeyForm").addEventListener("submit",
20   async function (event) {
21     event.preventDefault();
22     const privateKeyInput = document.getElementById("privateKeyInput")
23       .value.trim();
24     const passphraseInput = document.getElementById("passphrase")
25       .value;
26     if (!privateKeyInput) {
27       document.getElementById("privateKeyHelp").textContent = "
28       Inserisci la private key!";
29       return;
30     }
31     if (!passphraseInput || passphraseInput.length < 8) {
32       document.getElementById("privateKeyHelp").textContent = "La
33       passphrase deve avere almeno 8 caratteri!";
34       return;
35     }
36     // Convert private key string to Uint8Array (handle hex/base64)
37     let privateKeyUint8 = (isHex(privateKeyInput))
38       ? hexToUint8Array(privateKeyInput)
39       : base64ToUint8Array(privateKeyInput);
40     const encryptedObj = await encryptPrivateKey(privateKeyUint8,
41       passphraseInput);
42     const profile = await getProfile();
43     const walletAddress = normalizeWalletAddress(profile.utente.
44       wallet_addr);
45
46     await saveEncryptedKey(walletAddress, encryptedObj); // IndexedDB
47     storage keyed by walletAddress

```

```

40 privateKeyUint8.fill(0); // Zero memory
41 document.getElementById("privateKeySuccess").style.display = "
    block";
42 document.getElementById("privateKeyError").style.display = "none";
43 });

```

Listing 3: Client-side key encryption and storage workflow

The backend complements this flow by serving the `insert_private_key.html` page and exposing profile APIs that return the user's wallet address, which the frontend uses for storage keys. Importantly, all signing operations are performed client-side: when a transaction hash is prepared by the backend, the browser decrypts the locally stored encrypted key on-demand using the entered passphrase, uses the Circular SDK to produce a formatted signature securely, and immediately discards the raw key data from memory, reducing attack surface.

This architectural choice preserves strict key custody within the user agent, aligning with best practices in decentralized identity and cryptographic transaction signing, thereby maintaining the integrity of the supply-chain provenance assurance without compromising usability.

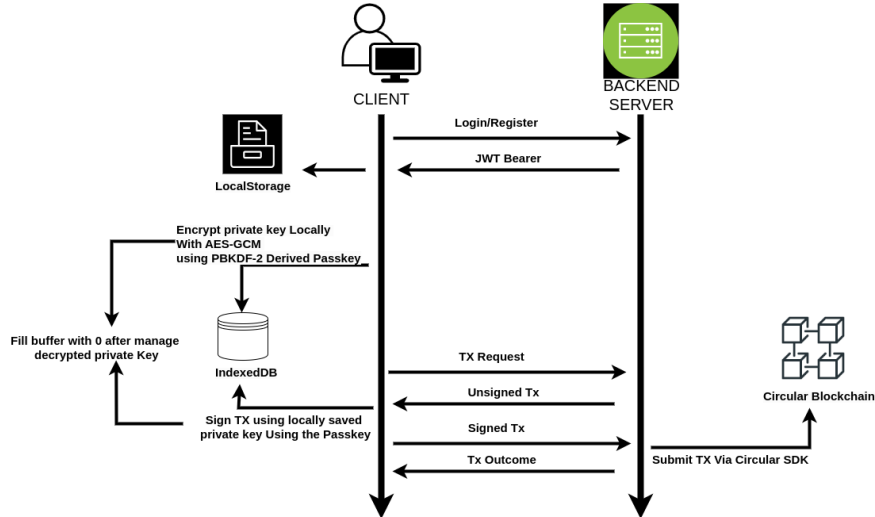


Figura 7: Private Key Management Workflow

## 6 Step Submission and Certificate Generation

The *Step* functionality enables users to record certified supply-chain events as blockchain transactions through a guided form interface. Users input batch-specific metadata including identifiers, product type, quantity, unit, location, timestamp, parental references, certification labels, and optional notes. This metadata forms the payload of a `C_TYPE_CERTIFICATE` transaction that anchors event provenance immutably on the Circular blockchain.

The submission process orchestrates a multi-phase sequence integrating client-side and backend responsibilities while preserving cryptographic integrity. The

frontend initiates the flow by packaging user inputs into a structured JSON object and issuing a request to the backend API endpoint responsible for transaction construction. The backend assembles an unsigned transaction as follows:

```

1 def define_transaction(blockchain, payload, sender, to):
2     circular = CircularProtocolAPI()
3     timestamp = helper.getFormattedTimestamp()
4     blockchain = helper.hexFix(blockchain)
5     nonce = int(circular.getWalletNonce(blockchain, sender)["Response"]
6                 ["Nonce"]) + 1
7     payload_hex = helper.hexFix(json.dumps(payload).encode().hex())
8
9     sender = helper.hexFix(sender)
10    to = helper.hexFix(to)
11    ID = str(blockchain + sender + to + payload_hex + str(nonce) +
12            timestamp)
13    hashID = helper.sha256(ID)
14
15    transactionType = "C_TYPE_CERTIFICATE"
16    data = {
17        'ID': hashID,
18        'From': sender,
19        'To': to,
20        'Timestamp': timestamp,
21        'Type': transactionType,
22        'Payload': payload_hex,
23        'Nonce': f"{nonce}",
24        'Signature': None,
25        'Blockchain': blockchain,
26        'Version': circular.getVersion()
27    }
28    return data, hashID

```

Listing 4: Backend logic for transaction construction

Upon receiving the unsigned transaction and transaction hash (hashID), the client prompts the user for their passphrase and retrieves the locally encrypted private key from IndexedDB.

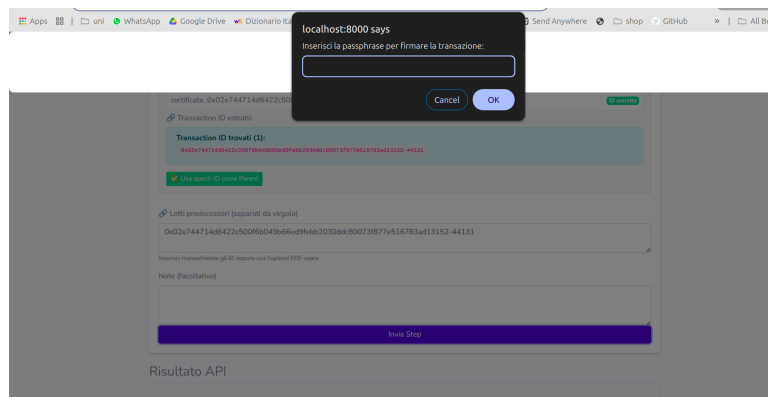


Figura 8: Passphrase used for retrieving Locally stored Private Key

This key is decrypted in-memory and used with the Circular Protocol SDK to produce a signature over the hashID. The signed transaction object is then submit-

ted back to the backend for broadcasting to the blockchain network. This multi-step client-sign flow ensures private keys never leave the user's environment without compromising transaction integrity or provenance authenticity.

Following the successful inclusion of a transaction on-chain, the system generates an audit-ready **certificate in PDF format**. Leveraging **fpdf**, the backend constructs a visually structured certificate that encapsulates the transaction metadata, decoded payload details, and cryptographic proofs, including a SHA-256 hash of the serialized certificate to ensure data integrity. An **optional QR code** representing the transaction ID and verification URL can be embedded in the certificate to facilitate convenient scanning and third-party verification. The certificate also plays a fundamental role in linking steps within the supply chain. Specifically, when defining a new step, the user can upload an existing certificate to associate the corresponding transaction as a "parent" operation, enabling traceability and end-to-end provenance across the supply chain workflow.

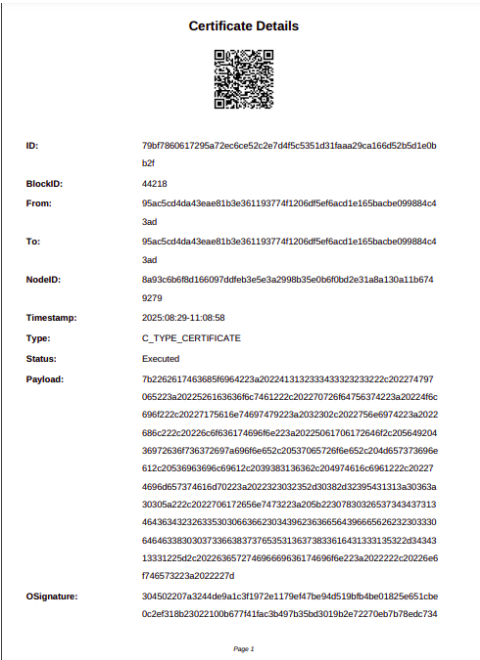


Figure 9: PDF Certificate Generated

This design ensures that transaction certification is not only cryptographically anchored on-chain but also human-accessible and verifiable, supporting auditors and downstream stakeholders in rigorous provenance validation workflows.

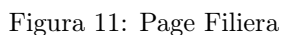
## 7 Activity Monitoring: Timeline

The *Timeline* feature provides authenticated users with a comprehensive, chronological view of all blockchain transactions associated with their wallet address. This interface captures both successfully executed and failed transactions, enabling users

Transaction statuses, including **Executed** and **Failed**, are used to set distinct visual cues, thus providing immediate feedback regarding the success or failure of blockchain anchoring operations.



The *Filiera* interface offers a chain-centric view of product provenance by reconstructing the sequence of supply-chain events associated with a particular transaction. This page serves as a pivotal tool for stakeholders—ranging from auditors to regulatory bodies—to validate the continuity and consistency of quantity, time-stamp, and location data across the entire certification graph.



13

in **QR codes** generated for every certified transaction and visually embedded within the corresponding digital certificates. Stored persistently in the MySQL database alongside base64-encoded QR images, these URIs provide a reliable mechanism for end-users and third parties to retrieve provenance data by simply scanning the QR code.

Upon invocation, the Filiera client-side script extracts blockchain and transaction identifiers from the URL context and issues authenticated requests to the backend API to fetch the transaction chain. The server responds with a structured representation of the target transaction and its ancestral steps, encoding the pertinent metadata.

The frontend decodes the hex-encoded payloads into human-readable JSON and presents them in an intuitive timeline. The design facilitates seamless traversal of dependencies, allowing inspectors to confirm that each chain segment adheres to operational and regulatory constraints.

A representative excerpt of the JavaScript handling data retrieval and display is as follows:

```

1 async function loadFilieraData(txId, blockchain) {
2   try {
3     const response = await fetch(`/api/filiera/chain/${txId}?
      blockchain=${blockchain}`);
4     if (!response.ok) {
5       throw new Error(`Failed to load chain data: ${response.
        statusText}`);
6     }
7     const data = await response.json();
8     displayProductInfo(data.current_transaction);
9     displayTimeline(data.supply_chain_timeline);
10  } catch (error) {
11    console.error('Error loading filiera data:', error);
12    showErrorMessage(error.message);
13  }
14 }

```

Listing 5: Filiera data loading and rendering logic

By anchoring provenance verification in accessible web interfaces augmented with blockchain-backed cryptographic assurances and QR-code facilitated retrieval, the Filiera page operationalizes transparency, traceability, and trust across the supply-chain ecosystem.

## 9 Conclusion and Future Developments

This work presents a comprehensive supply-chain traceability system leveraging the Circular protocol to achieve immutable provenance, decentralized consensus via Proof of Reputation, and user-centric cryptographic workflows. The integration of client-side secure key provisioning, interactive step submission, transparent activity timelines, and accessible provenance verification through the Filiera interface fosters robust data integrity and stakeholder trust across multi-organizational ecosystems.

The architecture harmonizes rigorous security controls, including ephemeral private key handling and JWT-based authentication, with usability considerations such

as QR-code facilitated certificate retrieval and human-readable payload decoding. Together, these components realize a practical, scalable framework aligning cutting-edge blockchain constructs with real-world regulatory compliance and operational workflows.

Future work will focus on several key areas to further enhance system robustness, scalability, and usability:

- **Definition Of A GetCoins Page:** At the moment is taken for granted that the user's registered wallet has some coins inside. These are needed for managing the fees related to the submission of the transaction related to the certification of the supply chain, in fact if the wallet is empty the transaction will fail.
- **Advanced Key Management:** Extend support for hardware-backed key storage to bolster both key security and user convenience, enabling safer and more user-friendly access for stakeholders.
- **User Experience Refinements:** Implement adaptive user interfaces that cater to various stakeholder roles, incorporate robust error handling, and automate key steps to optimize traceability workflows for greater efficiency and accuracy.
- **Governance and Policy Integration:** Integrate smart contract-driven policy enforcement, ensuring compliance with evolving regulations and adapting trust models as industry needs change.

By addressing these development avenues, the system aims to solidify its position as a foundational platform for trustworthy, transparent, and verifiable agri-food supply chain management, empowering organizations to meet increasing demands for accountability and data integrity through blockchain-enabled digital transformation.